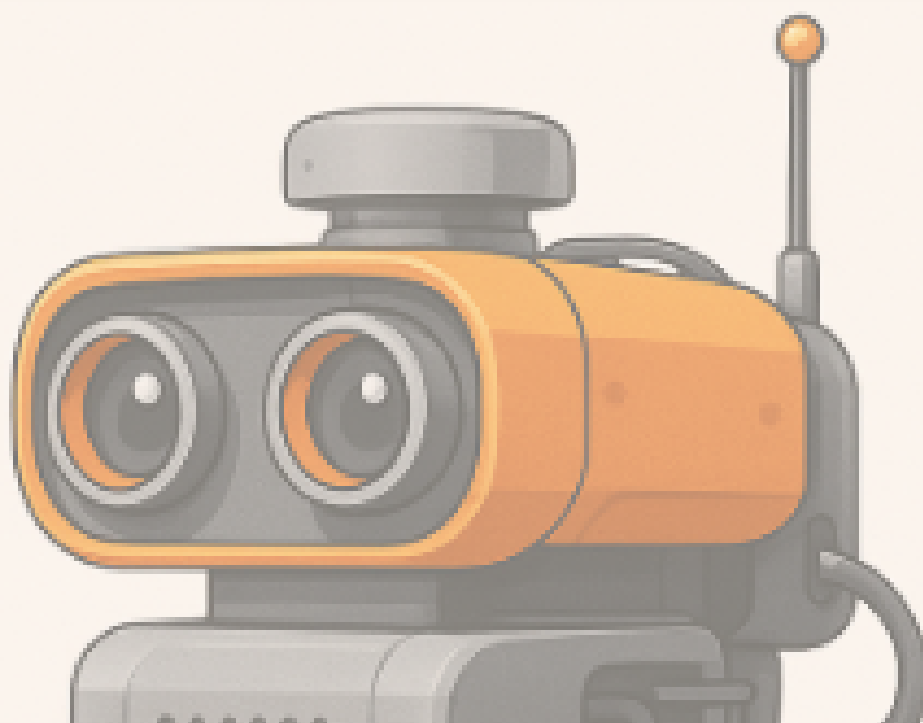


PROJET FIL ROUGE

Dossier de Spécifications



Réalisé par

- BACHAA Hajar
- BEN LTAIF Yasmine
- DEVAUD Antoine
- GRELET Thomas
- YAHYAOUI Nidal

Tuteur

- VANDERSTRAETEN
Julien
- Groupe n°3

Année universitaire 2025-2026

TABLE DES MATIÈRES

Introduction	04
---------------------	-----------

Description générale du projet	05
---------------------------------------	-----------

Fonctionnalités attendues	08
----------------------------------	-----------

1. Commande vocale	<i>P.08</i>
2. Commande textuelle	<i>P.09</i>
3. Traitement de l'image	<i>P.09</i>
4. Simulation	<i>P.11</i>

Spécifications détaillées	12
----------------------------------	-----------

1. Cas d'utilisation	<i>P.12</i>
2. Interfaces d'entrées et de sorties	<i>P.17</i>
3. Exemple de commandes utilisateur	<i>P.18</i>
4. Gestion des erreurs	<i>P.18</i>
5. Configuration et traçabilité	<i>P.22</i>

Gestion du projet	24
--------------------------	-----------

1. Organisation et planning	<i>P.24</i>
2. Outils et langages	<i>P.26</i>
3. Améliorations	<i>P.27</i>

Ouverture sur le projet	28
--------------------------------	-----------

TABLE DES FIGURES ET TABLEAUX

Figure 1 : Schéma fonctionnel du système de commande vocale et automatique	P.07
Figure 2 : Diagramme de séquence de traitement d'une commande vocale	P.08
Figure 3 : Diagramme de séquence du traitement de l'image	P.10
Figure 4 : Image d'une fenêtre <i>turtle</i>	P.11
Figure 5 : Diagramme de cas d'utilisation	P.12
Figure 6 : Schéma scénario 1, cas "cube violet trouvé"	P.15
Figure 7 : Schéma scénario 1, cas "cube violet non trouvé"	P.15
Figure 8 : Schéma scénario 2, cas "parcours effectué"	P.15
Figure 9 : Schéma scénario 2, cas "parcours impossible"	P.16
Tableau 1 : Description fonctionnelle du système	P.17
Tableau 2 : Exemples de commandes utilisateur	P.18
Tableau 3 : Gestion des erreurs et réactions attendues du robot	P.19
Tableau 4 : Gestion des erreurs liées à l'exécution des actions du robot	P.20
Tableau 5 : Gestion des erreurs liées au traitement d'images	P.20
Tableau 6 : Gestion des erreurs liées aux fichiers de configurations et de log	P.21
Tableau 7 : Gestion des erreurs de planification de l'exécution des missions	P.21
Figure 10 : Diagramme de séquence - traitement complet d'une requête utilisateur	P.23
Figure 11 : Diagramme de Gantt	P.24
Figure 12 : Vue sur la répartition des tâches sur Trello	P.27
Tableau 8 : Scénario de commandes personnalisées et interprétation système	P.28

TABLE DES ABRÉVIATIONS

IHM : Interface Homme Machine

NLU : Natural Language Understanding (Compréhension du Langage Naturel)

PFR : Projet Fil Rouge

PFR1 : Phase 1 du Projet Fil Rouge

PFR2 : Phase 2 du Projet Fil Rouge

STT : Speech To Text (Reconnaissance vocale)

Introduction

Le Projet Fil Rouge s'inscrit dans la formation de troisième année du parcours Systèmes Robotiques et Interactifs à l'UPSSITECH. Il constitue un projet intégrateur permettant de mettre en pratique les compétences acquises au cours du semestre, en particulier en algorithmique (Python), programmation impérative (C et Unix) et traitement de données (images et texte).

Ce projet est organisé en deux étapes complémentaires : la première, réalisée au cours du semestre 5, consiste à développer et valider des briques logicielles de base telles que la conversion de requêtes textuelles en commandes, la détection et la caractérisation d'objets dans des images, ainsi que la configuration et la simulation des déplacements d'un robot. Cette phase correspond essentiellement à un développement logiciel. La seconde étape, menée au semestre 6, vise à intégrer les modules développés dans la première partie sur une plateforme robotique réelle, avec des fonctionnalités plus avancées liées aux capteurs, au déplacement automatique et à la communication. Elle correspond à un développement complet, englobant à la fois les aspects matériels et logiciels.

L'équipe est constituée de cinq membres :

- BACHAA Hajar
- BEN LTAIF Yasmine
- DEVAUD Antoine
- GRELET Thomas
- YAHYAOUI Nidal

Notre tuteur, qui joue également le rôle de client, est VANDERSTRAETEN Julien.

Le présent document correspond au dossier de spécifications du PFRI. Il a pour objectif de définir de manière claire et détaillée les fonctionnalités attendues du logiciel, ses interactions avec l'environnement, les cas d'utilisation prévus, ainsi que les contraintes techniques et organisationnelles. Il constitue une référence essentielle pour guider la phase de développement et orienter les étapes ultérieures d'intégration et de validation.

Description générale du projet

Le Projet Fil Rouge repose sur la réalisation d'un système logiciel destiné à simuler les fonctionnalités essentielles d'une plateforme robotique. Cette plateforme, qui sera assemblée et testée dans la seconde partie du projet, doit à terme être capable d'interagir avec un utilisateur, de percevoir son environnement et d'adapter son comportement en conséquence. Dans le cadre du PFR1, l'objectif est de développer les briques logicielles de base qui serviront de fondations pour cette intégration future.

L'environnement technique s'appuie principalement sur le langage C dans un contexte Unix/Linux, conformément aux enseignements du semestre. Le langage Python est également mobilisé, notamment pour la simulation graphique des déplacements à l'aide de la bibliothèque *turtle* et pour certaines interactions comme la reconnaissance et la synthèse vocale. Cette combinaison permet d'assurer à la fois performance, modularité et interopérabilité.

Les acteurs du projet sont multiples. L'utilisateur interagit avec le système en formulant des commandes textuelles ou vocales, tandis que le robot simulé les exécute et fournit un retour visuel ou textuel. Le tuteur et l'équipe pédagogique jouent le rôle de clients, en fixant les exigences, en accompagnant le développement et en évaluant les livrables. Enfin, l'équipe projet elle-même est responsable de la conception, du développement et de l'intégration des modules logiciels.

Le périmètre du PFR1 est volontairement limité à la simulation logicielle : aucune intervention matérielle n'est prévue à ce stade. Les fonctionnalités doivent toutefois être conçues de manière générique et réutilisable pour faciliter leur intégration dans le PFR2. Les principales contraintes concernent le respect de l'environnement technique imposé, la modularité des solutions, la gestion des fichiers de configuration (langue, lexique, paramètres d'images) ainsi que la production de journaux (logs) pour assurer la traçabilité des résultats.

Afin de préparer l'intégration future sur la plateforme réelle (PFR2), le système développé dans le cadre du PFR1 doit déjà être capable de fonctionner selon deux logiques distinctes : le mode pilotage et le mode automatique.

Ces deux modes utilisent les mêmes briques logicielles (interpréteur, simulation, analyse d'images), mais diffèrent par l'origine de la commande et la manière dont le robot prend ses décisions.

Mode pilotage

Le mode pilotage correspond à une interaction directe entre l'utilisateur et le robot simulé. L'utilisateur contrôle les déplacements du robot en saisissant une commande textuelle ou vocale. Le système interprète cette requête, exécute la commande correspondante dans la simulation graphique, puis renvoie un retour visuel et/ou textuel. Toutes les actions sont enregistrées dans un fichier de logs afin d'assurer la traçabilité.

Ce mode a pour objectif de tester et valider le bon fonctionnement des modules d'interprétation et de simulation avant leur intégration matérielle. Il permet notamment de vérifier la compréhension des ordres, la gestion des erreurs et la cohérence entre les commandes et les mouvements affichés.

Mode automatique

Le mode automatique vise à simuler un comportement autonome du robot, sans intervention directe de l'utilisateur. Dans ce mode, le robot agit en fonction d'informations issues du module d'analyse d'images : il détecte des objets, identifie leurs caractéristiques, puis choisit une action adaptée selon des règles prédéfinies (par exemple : se diriger vers une balle rouge ou éviter un obstacle).

Le résultat de la décision est ensuite représenté visuellement dans la simulation, comme pour le mode pilotage.

Ce fonctionnement permet d'introduire une première logique de prise de décision et de préparer la gestion des situations réelles du PFR2, où le robot physique réagira à des capteurs et à des images captées en temps réel.

Schéma fonctionnel

Afin de mieux comprendre l'organisation générale du système, le fonctionnement global est représenté à l'aide d'un schéma fonctionnel (cf. Figure 1). Ce schéma met en évidence les différentes étapes du traitement d'une commande, qu'elle soit vocale ou automatique, ainsi que la gestion des erreurs et de l'arrêt du système.

Dans un premier temps, une phase d'initialisation est réalisée pour configurer le vocabulaire, les modules logiciels et les paramètres de fonctionnement. Ensuite, le système peut évoluer selon les deux modes suivants :

- **Mode Pilotage (P)**

- Le système attend la réception d'une commande vocale ou textuelle.
- La commande est analysée puis validée. Si elle est reconnue et réalisable, elle est exécutée (soit une action simple, soit une trajectoire).
- En cas de commande non reçue ou incomprise, le système retourne en veille ou active la gestion des erreurs.

- **Mode Automatique (A)**

- Le système attend la réception d'une image.
- L'analyse de l'image permet d'identifier des objets, couleurs ou positions afin de déterminer si le chemin est libre.
- Si le chemin est disponible, l'action est validée et exécutée (avancer ou tourner). Sinon, la gestion des erreurs est activée.

Une gestion centralisée des erreurs intervient en cas de commande impossible ou d'obstacle détecté, afin d'assurer la sécurité et la continuité du fonctionnement.

Enfin, une commande d'arrêt ou la fin du parcours entraîne l'arrêt complet du système.

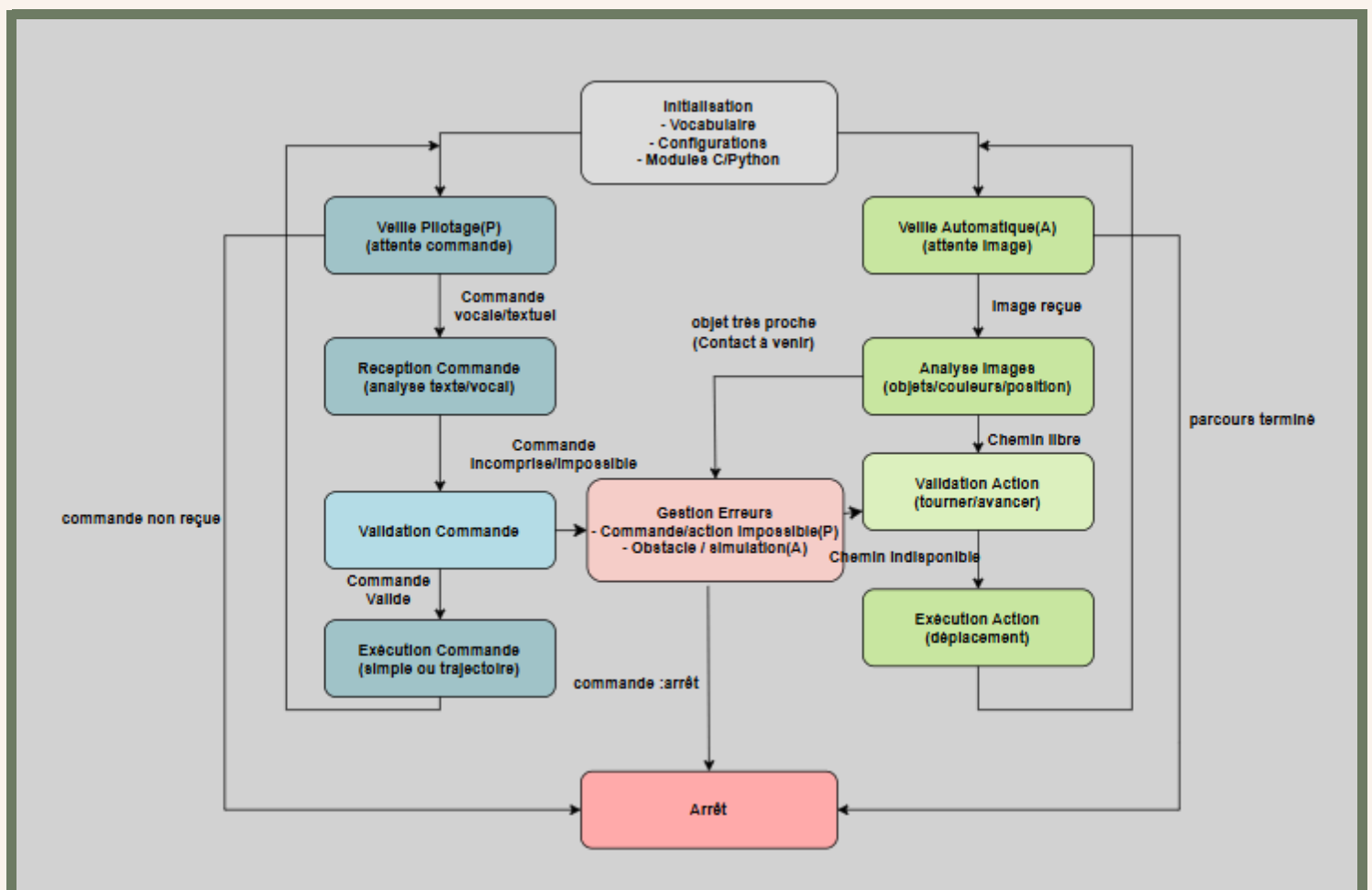


Figure 1 : Schéma fonctionnel du système de commande vocale et automatique

Fonctionnalités attendues

Les fonctionnalités attendues du système dans le cadre de la première phase du projet peuvent être regroupées en quatre grands volets.

1 Commande vocale

- Exploiter un module de reconnaissance vocale pour transcrire les requêtes orales de l'utilisateur en requêtes texte.
- Analyser et transformer ces requêtes texte en requêtes commandes interprétables et exécutables par le robot.
- Fournir un retour à l'utilisateur en cas d'impossibilité d'exécution d'une requête commande (exemple : « je n'ai pas trouvé de balle rouge »).
- Permettre la synthèse vocale ou l'émission de signaux sonores comme réponses du système afin de confirmer ou d'expliquer le résultat de la requête.

Le diagramme de séquence suivant montre les différentes étapes du traitement d'une commande vocale.

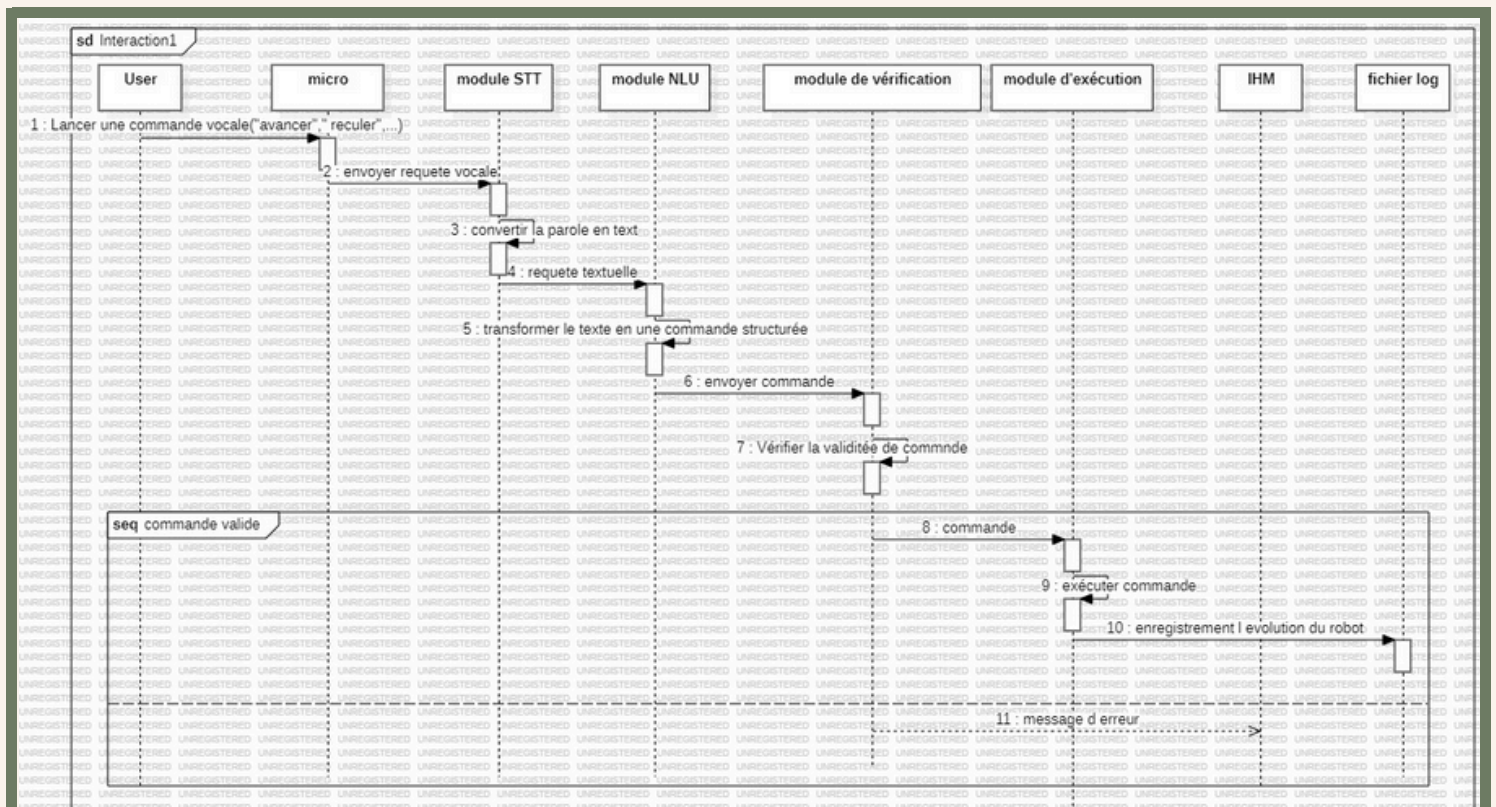


Figure 2 : Diagramme de séquence de traitement d'une commande vocale

2 Commande textuelle

- Interpréter une requête formulée par l'utilisateur (en français, anglais ou espagnol). Le robot doit comprendre ces dernières pour ainsi effectuer les tâches.
- Associer cette requête à une commande exécutable par le robot simulé.
- Gérer des paramètres comme les distances ou les couleurs d'objets.
- Prendre en compte des formulations variées grâce à des fichiers de configuration (lexique, synonymes, langues). Donc en soit reconnaître le vocabulaire et comprendre la structure de la phrase.
- Prévoir la gestion des erreurs (exemple : commande non reconnue).

3 Traitement de l'image

Le module de traitement d'image a pour objectif de détecter et caractériser les objets présents dans une image à partir de leurs caractéristiques visuelles :

- la couleur dominante
- la forme géométrique
- leur position ou taille

Ce module constitue la base du futur système de vision du robot, et sera réutilisé dans le PFR2 pour l'analyse d'images en temps réel (caméra embarquée).

Principe général

L'image d'entrée est fournie sous la forme de fichiers contenant les matrices de couleur. Chaque pixel contient trois composantes de couleur : Rouge (R), Vert (G) et Bleu (B). Le traitement s'effectue en plusieurs étapes :

1. Lecture et conversion de l'image

- a. Chargement du fichier image dans une structure de données (tableau 2D de pixels).
- b. Conversion éventuelle en niveaux de gris ou en format simplifié pour faciliter l'analyse.

2. Analyse de la couleur

- a. Quantification sur 2, 3 ou 4 bits par couleur
- b. Calcul d'un histogramme des couleurs : pour chaque pixel, on incrémente la fréquence de sa composante dominante (R, G, B).
- c. Détermination de la couleur prédominante de l'objet.
- d. Utilisation de seuil de détection pour reconnaître des couleurs simples (rouge, vert, bleu, jaune...).

3. Analyse de la forme

- L'image est d'abord binarisée : les pixels appartenant à l'objet sont marqués en blanc (1), le reste en noir (0).
- Application d'un algorithme de détection de contours (par différence de pixels voisins).
- Comptage du nombre de bords et analyse des proportions pour reconnaître la forme :
 - 3 côtés → triangle
 - 4 côtés → rectangle/carré
 - contours arrondis → cercle

4. Récupération des coordonnées

- Calcul du centre de gravité ou de la boîte englobante de l'objet.
- Ces coordonnées (x, y) seront renvoyées à la partie "Commande du robot".

5. Enregistrement des résultats

- Les informations extraites (forme, couleur, coordonnées) sont enregistrées dans un fichier de log (resultats.txt ou .csv).
- Exemple de sortie :
 - Objet 1 : cercle rouge - position (120, 250)
 - Objet 2 : rectangle bleu - position (310, 180)

Le diagramme de séquence suivant montre les différentes étapes du traitement d'une image.

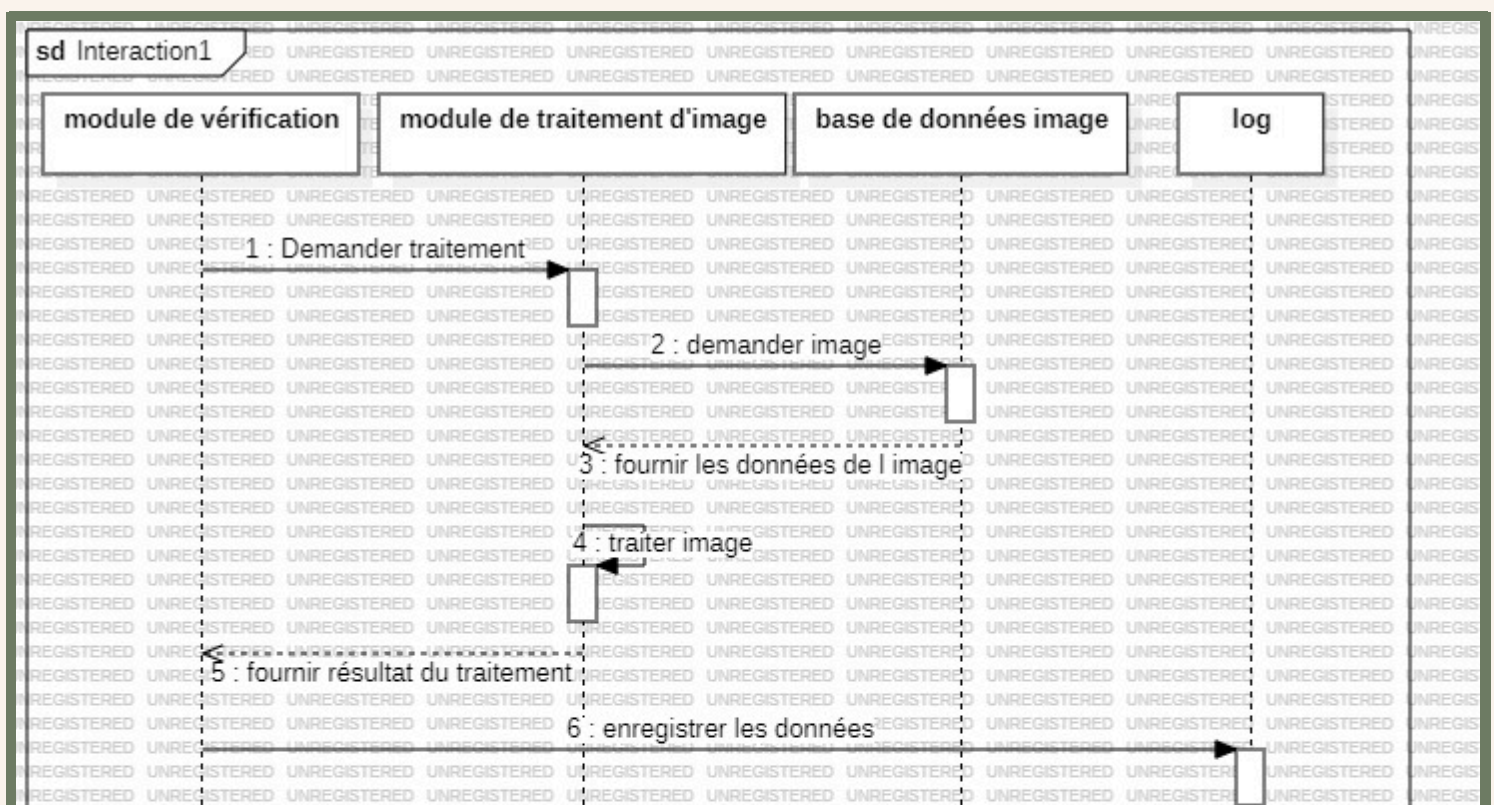


Figure 3 : Diagramme de séquence du traitement de l'image

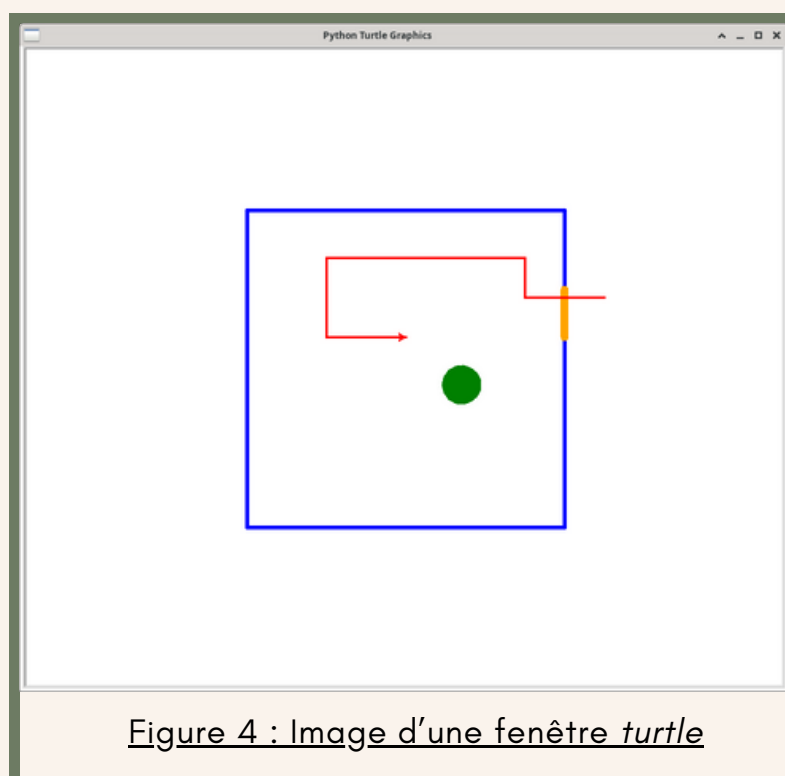
Contraintes techniques

- Le module doit être développé en langage C sous un environnement Unix.
- L'utilisation de bibliothèques externes de vision (OpenCV, etc.) est interdite.
- Les algorithmes doivent être simples et explicites (parcours de pixels, filtrage basique, calculs de moyenne, etc.).
- Le code doit être commenté, modulaire et réutilisable dans le PFR2.
- Jeu d'images fourni par l'équipe pédagogique (formes et couleurs variées).
- Tests unitaires pour vérifier :
 - Détection correcte de la couleur dominante,
 - Reconnaissance correcte de la forme,
 - Exactitude des coordonnées calculées.

4 Simulation

- Utiliser la bibliothèque *turtle* de Python pour représenter les déplacements.
- Gérer les commandes basiques (avancer, reculer, tourner, s'arrêter).
- Prendre en charge des trajectoires simples ou complexes (cercle, carré, zigzag...).
- Prévoir la possibilité d'enchaîner plusieurs commandes dans une seule requête.

L'image ci-contre permet de visualiser un exemple d'interface graphique avec *turtle*, sur laquelle est représentée une pièce en **bleu**, une ouverture en **jaune**, un obstacle en **vert** et le déplacement du robot en **rouge**. La "tortue" représente le robot.



Spécifications détaillées

1 Cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous illustre les principales interactions entre l'utilisateur et le système du robot commandé. Il permet de représenter de manière claire les différentes fonctionnalités offertes par le système ainsi que les modes de commande disponibles, conformément aux spécifications du cahier des charges.

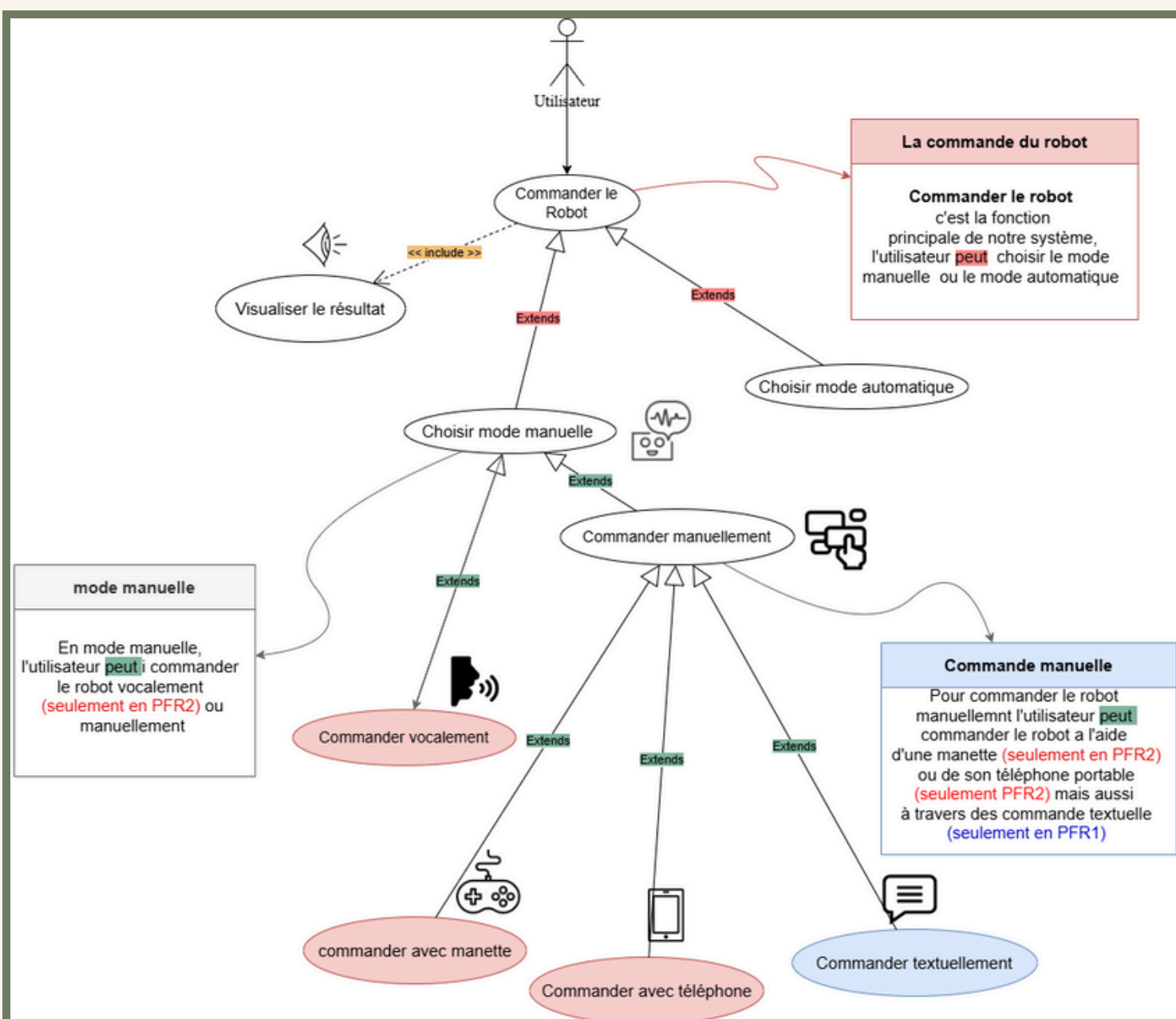


Figure 5 : Diagramme de cas d'utilisation

Cas général

1. Acteur principal

L'acteur principal du système est l'utilisateur. Il interagit directement avec le robot pour le commander, choisir un mode de fonctionnement, et visualiser les résultats des actions exécutées. C'est lui qui initie toutes les opérations de contrôle du robot.

2. Cas d'utilisation principal : Commander le robot

Le cas d'utilisation « Commander le robot » représente la fonction principale du système. Il regroupe toutes les actions nécessaires pour contrôler le robot, que ce soit en mode manuel ou en mode automatique.

Ce cas d'utilisation inclut également la possibilité de visualiser le résultat, c'est-à-dire d'observer le comportement du robot ou les retours d'exécution du système.

- Relation **<<include>>** :

Le cas Visualiser le résultat est inclus, car à chaque commande exécutée, le système affiche ou enregistre le résultat de l'action.

- Relation **<<extends>>** :

Les cas Choisir mode manuelle et Choisir mode automatique étendent le cas principal, car ils constituent des sous-fonctionnalités permettant de sélectionner le type de commande souhaité.

3. Mode automatique

Dans le mode automatique, le robot exécute des tâches de manière autonome, sans intervention directe de l'utilisateur. Ce mode repose sur la compréhension d'un objectif donné et la prise de décision automatique selon les données captées par la caméra et les capteurs. A noter que cette fonctionnalité est prévue pour la phase PFR2 du projet.

4. Mode manuelle

Le mode manuel permet à l'utilisateur de prendre le contrôle direct du robot. Dans ce mode, le robot exécute uniquement les commandes envoyées par l'utilisateur à travers différents moyens d'interaction.

Le cas d'utilisation Choisir mode manuelle est relié à Commander manuellement, qui se décline en plusieurs sous-cas correspondant aux différentes méthodes de commande :

a. Commande textuelle (PFR1)

Ce mode de commande, implémenté dès la première phase du projet (PFR1), permet à l'utilisateur d'envoyer des instructions textuelles au robot via un terminal ou une interface (exemples : "forward", "left", "stop"). Il constitue le mode de base du système, utilisé pour les premiers tests de communication entre le robot et l'utilisateur.

b. Commande vocale (PFR1)

Ce mode, prévu pour la phase PFR1, permettra à l'utilisateur de commander le robot (simulé au PFR1 ou physique au PFR2) par la voix grâce à un module de reconnaissance vocale. L'objectif est d'améliorer l'ergonomie et la facilité d'utilisation du robot en offrant un contrôle mains libres.

c. Commande avec manette (PFR2)

Dans ce mode, l'utilisateur contrôle le robot à l'aide d'une manette filaire ou Bluetooth. Ce mode vise à offrir un contrôle précis et intuitif du robot, similaire à une télécommande.

d. Commande avec téléphone (PFR2)

Ce mode, également prévu pour la phase PFR2, permettra à l'utilisateur de piloter le robot via une application mobile ou une interface web connectée à la Raspberry Pi. Les commandes seront transmises sans fil, ce qui améliore la portabilité et la flexibilité du système.

5. Visualiser le résultat

Le cas d'utilisation « Visualiser le résultat » est inclus dans le cas principal. Il permet à l'utilisateur d'observer le comportement du robot après l'envoi d'une commande, ou de consulter les résultats enregistrés (par exemple, les journaux d'exécution ou les retours visuels de la caméra). Cette fonction est essentielle pour valider le bon fonctionnement du robot et pour analyser les performances du système.

Exemples de cas d'utilisation

Afin d'illustrer le fonctionnement du mode de commande vocale, plusieurs scénarios d'utilisations peuvent être envisagés, allant de commandes simples à des consignes plus complexes selon la situation.

Parmi ces possibilités, deux exemples de situations ont été décrits ci-dessous afin de représenter, pour chacune, deux cas différents. Ces exemples permettent de mieux comprendre la logique de fonctionnement et les réactions du robot en fonction des ordres reçus.

Situation 1 : Aller jusqu'au cube violet

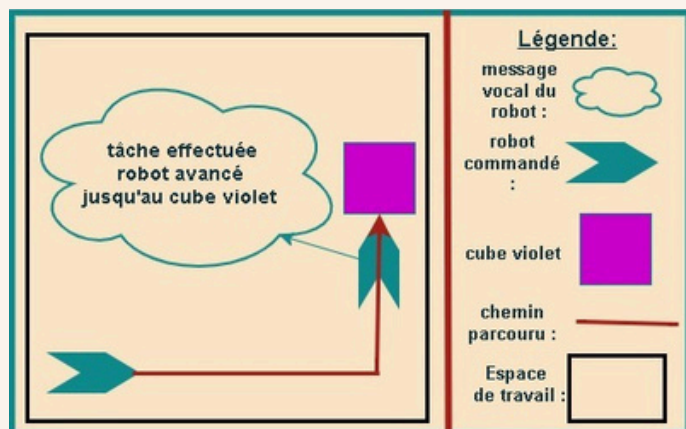


Figure 6 : Schéma scénario 1, cas "cube violet trouvé"

Commande vocale :

« Avance jusqu'au cube violet. »

Explications :

Le robot interprète correctement la commande, se déplace jusqu'à détecter le cube violet et s'arrête. Le schéma montre la trajectoire parcourue et un message de confirmation : « Tâche effectuée, robot avancé jusqu'au cube violet. »

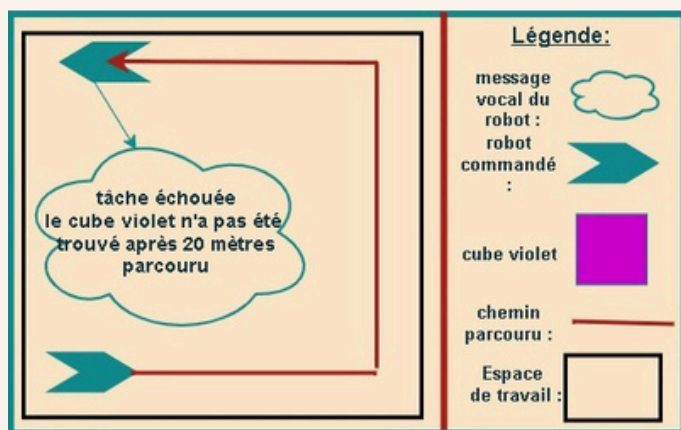


Figure 7 : Schéma scénario 1, cas "cube violet non trouvé"

Commande vocale :

« Avance jusqu'au cube violet. »

Explications :

Dans ce cas, le robot exécute la commande mais ne parvient pas à détecter le cube violet après avoir parcouru la distance maximale prévue de 20 mètres (paramètre prédéfinis auparavant). Le schéma montre la trajectoire effectuée et le message associé : « Tâche échouée, le cube violet n'a pas été trouvé après 20 mètres parcourus. » Cette situation illustre une limite de détection ou une erreur de reconnaissance de l'objet.

Situation 2 : Avancer d'une distance de 3 mètres

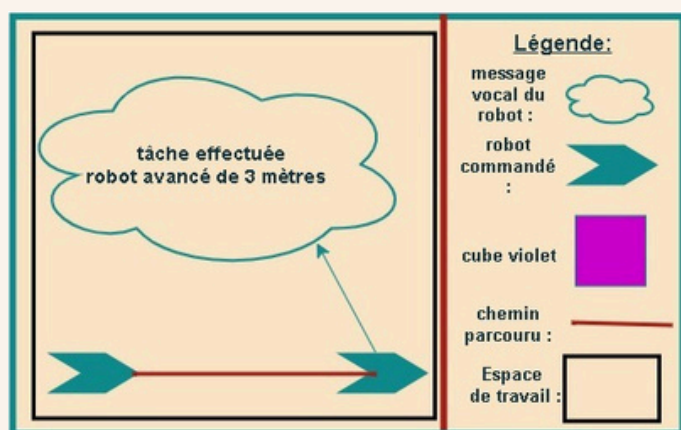


Figure 8 : Schéma scénario 2, cas "parcours effectué"

Commande vocale :

« Avance de trois mètres en ligne droite. »

Explications :

Le robot comprend la commande et parcourt exactement la distance demandée.

Le déplacement est représenté par une flèche proportionnelle à la distance parcourue, accompagnée du message : « Tâche effectuée, robot avancé de trois mètres. »

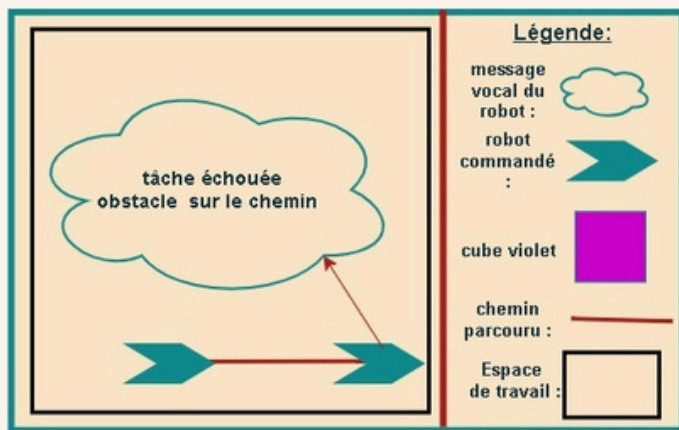


Figure 9 : Schéma scénario 2, cas "parcours impossible"

Commande vocale :

« Avance de trois mètres en ligne droite. »

Explications :

Lors de la même commande, le robot rencontre un obstacle sur sa trajectoire avant d'avoir atteint les 3 mètres. Le schéma illustre la flèche interrompue par l'obstacle (mur), ainsi que le message correspondant : « Tâche échouée, obstacle sur le chemin. »

Cette représentation met en évidence la gestion des imprévus et la prise en compte de la sécurité dans le déplacement du robot.

Synthèse

Ces représentations constituent des exemples de situations parmi l'ensemble des cas possibles que le robot peut rencontrer lors d'un pilotage vocal.

Elles montrent concrètement comment le robot réagit selon la commande reçue et les conditions de son environnement.

Les cas de réussite et d'échec permettent de visualiser le fonctionnement global du système, d'en comprendre les limites et d'évaluer la fiabilité de la commande vocale dans différentes configurations d'utilisation.

2 Interface d'entrées et de sorties

Entrées possibles

- Requêtes textuelles saisies par l'utilisateur.
- Requêtes vocales, transcrites en texte via le module fourni.
- Images fixes issues d'un jeu d'images prédéfini utilisé pour l'analyse.
- Fichiers de configuration définissant la langue, le lexique ou les paramètres d'analyse d'images.

Sorties générées

- Commandes interprétées et exécutées par le robot simulé.
- Représentation graphique des déplacements à l'aide de *turtle*.
- Résultats du traitement d'images (exemple : coordonnées d'objets détectés, caractéristiques de couleur ou de taille).
- Fichier de logs retraçant les commandes exécutées, les résultats obtenus et les erreurs rencontrées.

Le tableau suivant présente une synthèse des principales fonctionnalités prévues dans le système, leur objectif ainsi que leurs entrées et sorties associées.

Fonctionnalité	Fonction	Objectif de la fonction	Entrée(s)	Sortie(s)
Commande Vocale	Transcrire un son	Transcrire une commande vocale en requête texte	Commande vocale (audio)	Requête (texte)
	Analyser du texte	Comprendre une requête texte pour en déduire l'action demandée	Requête (texte)	Commande réelle (texte)
Traitement d'images	Identifier une forme	Déduire la forme d'un élément sur une image	Image de l'environnement	Forme
	Identifier une couleur	Déduire la couleur d'un élément sur une image	Image de l'environnement	Couleur
	Identifier une position	Déduire la position d'un élément sur une image	Image de l'environnement	Position
Lien utilisateur - robot	Action de l'utilisateur vers le robot	Traduire l'action de l'utilisateur en commande robot	Commande de l'utilisateur	Commande pour le robot
	Action du robot vers l'utilisateur	Informar l'utilisateur des actions du robot	Action du robot	Affichage pour l'utilisateur
Simulation	Passer en mode pilotage	Afficher un menu pour contrôler le robot	Environnement, Choix de l'utilisateur	Affichage pour l'utilisateur
	Passer en mode autonome	Lancer le robot dans un environnement prédéfini	Environnement	Affichage pour l'utilisateur

Tableau 1 : Description fonctionnelle du système

3 Exemples de commandes utilisateur

Le tableau suivant présente les principaux types de commandes que l'utilisateur peut saisir ou prononcer, leur interprétation par le système ainsi que l'action correspondante réalisée par le robot simulé. Ces exemples illustrent les fonctions de déplacement, de détection d'objets et de changement de configuration linguistique.

Type de commande	Exemple saisi ou prononcé	Interprétation par le système	Action du robot simulé
Déplacement simple	« avance de deux mètres »	Conversion en commande Go_forward(2)	Le robot avance de 2 mètres vers l'avant.
Rotation	« tourne à gauche »	Conversion en commande Turn_left(90)	Conversion en commande Turn_left(90) Le robot pivote de 90° vers la gauche.
Déplacements combinés	« avance de trois mètres puis tourne à droite »	Séparation en deux commandes : Go_forward(3) et Turn_right(90)	Le robot avance puis effectue un virage à droite.
Arrêt	« arrête-toi »	« arrête-toi » Commande Stop()	Le robot cesse tout mouvement immédiatement.
Détection d'objets	« cherche la balle rouge »	Appel du module d'analyse d'image	Le système identifie la balle rouge et affiche sa position.
Changement de langue	« passe en anglais »	Modification de la configuration linguistique	Le système recharge les fichiers de configuration en anglais.

Tableau 2 : Exemples de commandes utilisateur

4 Gestion des erreurs

Dans tout projet logiciel ou robotique, la gestion des erreurs et des défaillances est une composante essentielle de la conception. Elle garantit non seulement la fiabilité du système, mais aussi sa robustesse face aux situations imprévues.

Un dossier de spécifications qui décrit les erreurs possibles et la manière dont elles sont traitées démontre la maturité technique et la préparation opérationnelle de l'équipe projet. Anticiper les erreurs permet d'éviter que le système ne réagisse de manière incohérente ou dangereuse.

En définissant dès le départ les cas d'erreurs possibles (entrées invalides, absence d'objets détectés, échec d'exécution...), on garantit un comportement contrôlé plutôt qu'imprévisible.

Un système fiable est un système capable de résister aux défaillances.

Dans un projet de robotique, une erreur non gérée peut conduire à un mouvement dangereux, une perte de données ou un blocage complet du programme.

Documenter les erreurs possibles permet donc de :

- détecter rapidement l'origine du problème,
- éviter les comportements imprévisibles,
- assurer la sécurité des utilisateurs et du matériel.

Enfin, dans le Projet Fil Rouge, anticiper les défaillances dès le PFR1 (simulation logicielle) simplifie énormément l'intégration dans le PFR2 (robot réel).

Les modules déjà capables de détecter et signaler leurs erreurs seront plus faciles à adapter aux conditions réelles : capteurs défaillants, bruits parasites, obstacles non prévus, etc.

Toutes les erreurs sont répertoriées ci-après.

1. Commandes vocales et textuelles

Le tableau 3 présente les principales catégories d'erreurs susceptibles de se produire lors de l'interprétation des commandes utilisateur, ainsi que les réactions attendues du robot pour chaque situation identifiée.

Catégorie	Cause probable	Réaction attendue du robot
Commande inconnue	Mot absent du lexique ou phrase non comprise	Afficher ou dire "Commande inconnue." et ignorer la commande
Commande ambiguë	Plusieurs objets ou directions possibles	Demander une précision : "Lequel ? La balle rouge ou la bleue ?"
Mauvaise syntaxe	Erreur dans la phrase	Rejeter la commande et demander une reformulation
Langue non reconnue	Utilisateur parle une autre langue	Répondre : "Langue non reconnue. Langue actuelle : français."
Donnée manquante	Valeur absente ("avance de...")	Utiliser une valeur par défaut et prévenir l'utilisateur

Tableau 3 : Gestion des erreurs et réactions attendues du robot

2. Simulation de déplacement

Le tableau 4 présente les principales catégories d'erreurs pouvant survenir lors de l'exécution des actions du robot, ainsi que les réactions attendues pour garantir la sécurité, la cohérence et la continuité du fonctionnement.

Catégorie	Cause probable	Réaction attendue du robot
Collision virtuelle	Rencontre d'un mur ou d'une limite	Stopper le mouvement et afficher "Obstacle détecté."
Trajectoire impossible	Commande incohérente	Ignorer la commande et signaler l'erreur
Mouvement infini	Boucle sans fin	Forcer un arrêt automatique
Commandes contradictoires	Ordres opposés	Exécuter les deux et noter l'incohérence dans le log
Trop de commandes successives	Entrées trop rapides	Mettre une pause entre chaque commande

Tableau 4 : Gestion des erreurs liées à l'exécution des actions du robot

3. Analyse d'images

Le tableau 5 présente les principales catégories d'erreurs susceptibles de se produire lors du traitement des images, ainsi que les réactions attendues du robot pour garantir une interprétation fiable des données visuelles.

Catégorie	Cause probable	Réaction attendue du robot
Aucun objet détecté	Image vide ou floue	Afficher "Aucun objet trouvé."
Plusieurs objets similaires	Objets identiques détectés	Choisir le plus proche ou demander une précision
Mauvaise détection	Faux positif	Continuer et noter l'erreur dans le log
Couleur non reconnue	Valeur hors plage	Ignorer la zone et signaler l'erreur
Image illisible	Fichier manquant / corrompu	Afficher "Erreur de lecture d'image."

Tableau 5 : Gestion des erreurs liées au traitement d'images

4. Fichiers de configurations et logs

Le tableau 6 présente les principales catégories d'erreurs liées aux fichiers de configuration et de journalisation (log), ainsi que les réactions attendues du robot pour assurer une exécution stable malgré les anomalies détectées.

Catégorie	Cause probable	Réaction attendue du robot
Fichier de configuration absent	Fichier introuvable	Charger les valeurs par défaut et avertir
Format de fichier invalide	Erreur dans la syntaxe du fichier	Ignorer la ligne et loguer l'erreur
Paramètre incohérent	Valeur impossible	Utiliser la valeur précédente et noter l'anomalie
Fichier de log inaccessible	Droits d'écriture ou chemin invalide	Continuer sans log mais avertir l'utilisateur

Tableau 6 : Gestion des erreurs liées aux fichiers de configurations et de log

5. Logique automatique et décision

Le tableau 7 présente les principales catégories d'erreurs susceptibles de survenir lors de la planification ou de l'exécution d'une mission, ainsi que les réactions attendues du robot pour assurer un comportement cohérent malgré les situations ambiguës ou non réalisables.

Catégorie	Cause probable	Réaction attendue du robot
Aucune action possible	Mission irréalisable	S'arrêter et afficher "Aucune trajectoire possible."
Objet recherché absent	"Trouve la balle bleue" mais rien trouvé	Répondre : "Je ne trouve pas la balle bleue."
Conflit d'objectifs	Ordres incompatibles	Choisir une priorité (éviter obstacle d'abord) et loguer la décision
Requête trop complexe	Phrase trop longue ou ambiguë	Exécuter la première partie et ignorer le reste

Tableau 7 : Gestion des erreurs de planification de d'exécution des missions

5 Configuration et traçabilité

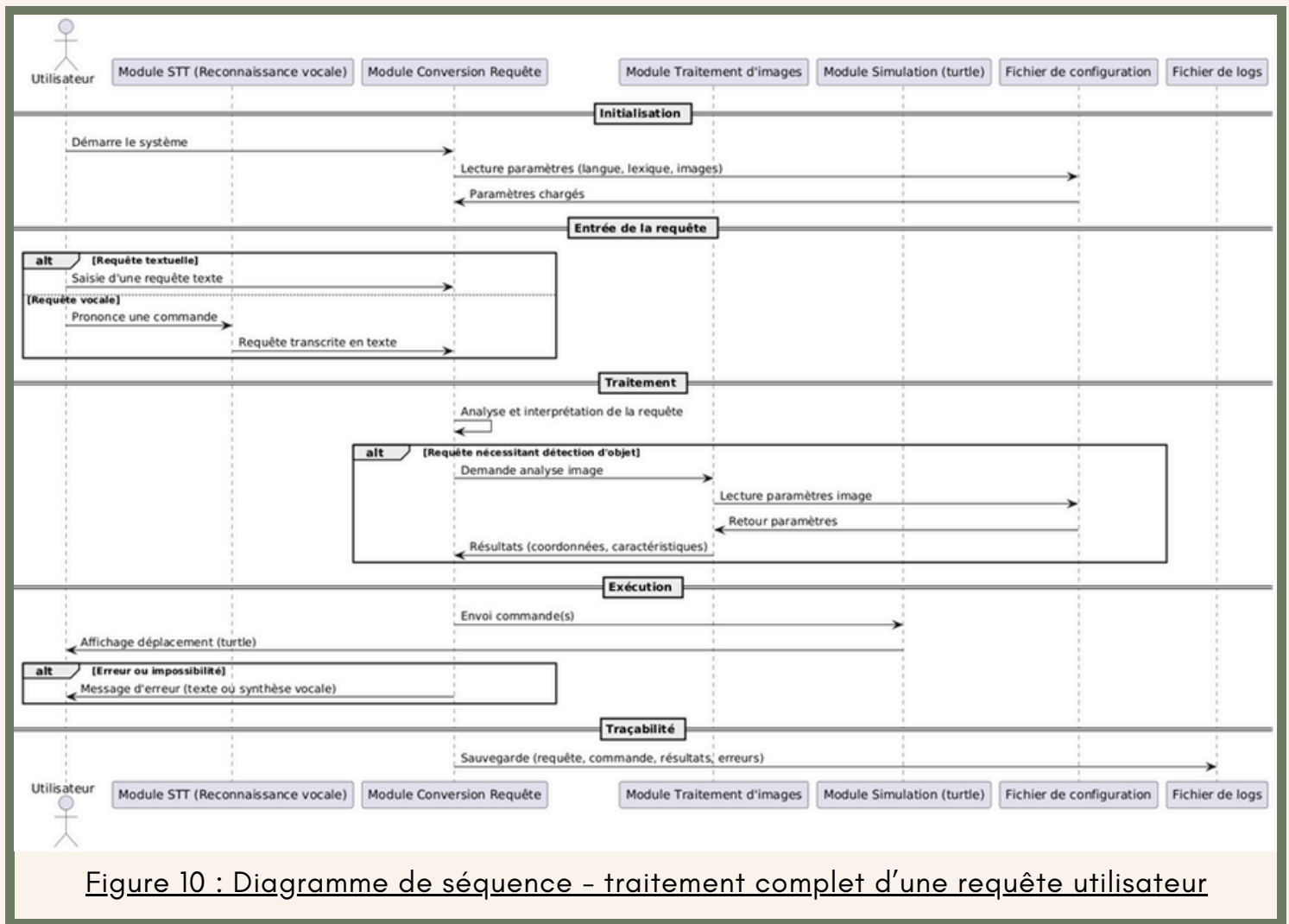
La configuration et la traçabilité assurent la flexibilité et la fiabilité du système.

- Personnalisation des fonctionnalités :
 - Les fichiers de configuration permettent d'adapter facilement le logiciel sans modifier le code source.
- Les éléments configurables incluent notamment :
 - Langue d'utilisation : possibilité de sélectionner la langue principale de communication entre français, anglais ou espagnol pour les requêtes textuelles et vocales.
 - Lexique des commandes : ajout, suppression ou modification de mots-clés et de synonymes associés aux actions (par exemple : ajouter "va tout droit" comme équivalent de "avance").
 - Paramètres de détection d'images : définition des couleurs à reconnaître (rouge, vert, bleu, jaune), des formes à détecter (cercle, rectangle, triangle), des niveaux de quantifications (2, 3 ou 4 bits par couleur), ainsi que des seuils de tolérance pour la reconnaissance (par exemple intensité minimale de couleur).
 - Distance maximale que le robot pourra parcourir avant arrêt du système (cf. Figure 7, où l'exemple était de 20 mètres).

Cette modularité rend le système plus flexible et facilite sa maintenance ou son évolution.

- Fichier de logs :
 - Le programme génère un fichier retraçant les actions réalisées, les résultats obtenus et les erreurs rencontrées.
 - Ces traces facilitent le débogage, la validation des tests et garantissent la traçabilité du projet. Le format retenu pour ce module est JSON, ce qui permet une structuration claire et une exploitation aisée des informations.
- Réutilisabilité des modules :
 - Grâce à cette structure claire, les briques logicielles pourront être réutilisées et intégrées facilement dans le PFR2.
 - Cette approche assure une continuité entre la simulation logicielle et le fonctionnement du robot réel.

Le diagramme de séquence ci-dessous illustre les interactions entre les différents modules du système depuis la réception d'une requête jusqu'à son exécution et sa journalisation (cf. Figure 2). Il met en évidence les échanges successifs entre les modules de reconnaissance vocale, de conversion de requête, de traitement d'images, de simulation, ainsi que les fichiers de configuration et de logs.



Ce diagramme présente les étapes suivantes :

- **Initialisation** : chargement des paramètres nécessaires (langue, lexique et images).
- **Entrée de la requête** : l'utilisateur saisit une requête textuelle ou prononce une commande vocale, transcrite en texte.
- **Traitement** : la requête est interprétée et, si nécessaire, une analyse d'image est déclenchée pour détecter des objets et récupérer leurs coordonnées.
- **Exécution** : les commandes sont envoyées à la simulation pour afficher le déplacement. En cas d'erreur ou d'impossibilité d'exécution, un message est renvoyé à l'utilisateur.
- **Traçabilité** : les informations relatives à la requête, aux résultats et aux erreurs sont enregistrées dans le fichier de logs.

Gestion du projet

L'organisation du projet repose à la fois sur une planification rigoureuse des différentes étapes et sur une répartition équitable des tâches au sein de l'équipe. Elle s'appuie également sur des outils collaboratifs permettant de faciliter le suivi et la communication.

1 Organisation et planning

Calendrier global

Le calendrier global du PFR1 est défini par l'équipe pédagogique et comprend plusieurs jalons importants :

- Semaine du 15 septembre 2025 : mise à disposition du cahier des charges et constitution des groupes.
- 17 septembre et 22 septembre 2025 : présentation générale du projet et validation des équipes.
- 22 octobre 2025 : rendu du dossier de spécifications incluant le planning et la répartition des tâches.
- Début novembre à mi-décembre 2025 : phase de conception détaillée, développement des modules et tests unitaires.
- Début janvier 2026 : intégration des différentes briques logicielles et préparation à la validation.

Semaine du 19 janvier 2026 : validation finale du projet (démonstration et dépôt des livrables).

Planning prévisionnel

Le diagramme de Gantt suivant permet de planifier les différentes fonctionnalités à intégrer en fonction du temps. Il correspond à la phase de développement et de tests uniquement.

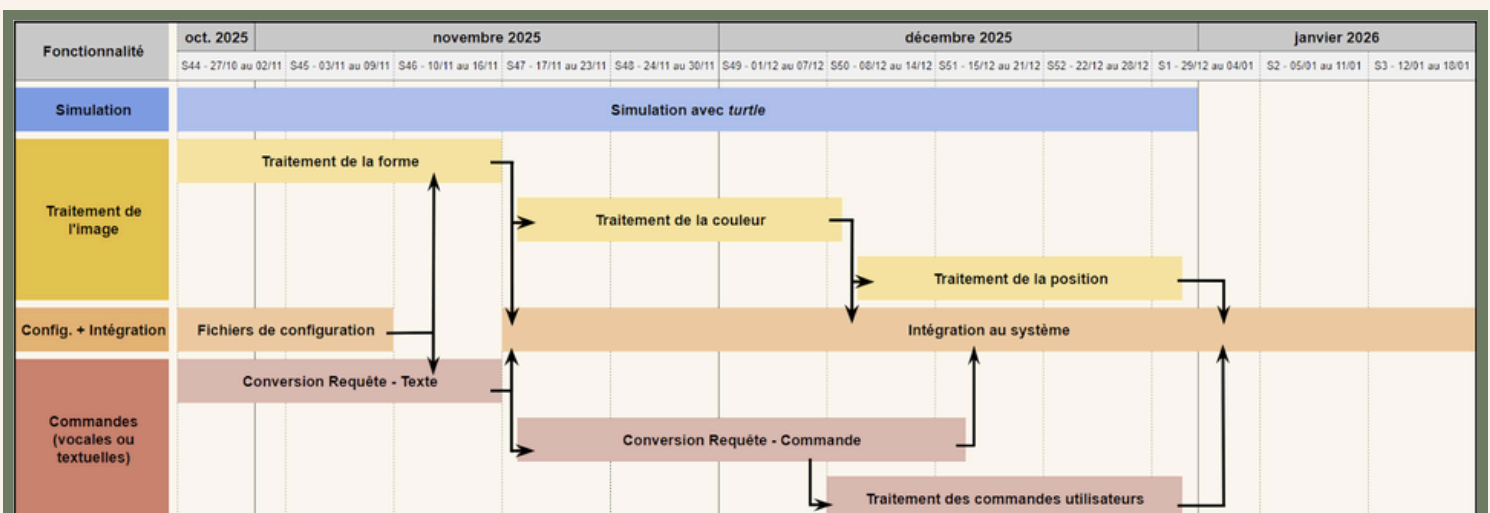


Figure 11 : Diagramme de Gantt

Répartition des tâches

La répartition des tâches devra suivre une contrainte claire : tous les membres du groupe n'ont pas les mêmes compétences en programmation. Ainsi, on a :

- Connaissances basiques : Antoine, Hajar et Yasmine
- Connaissances avancées : Nidal et Thomas

Une répartition peut alors être proposée, comme suit :

- Simulation : Hajar
- Traitement de l'image : Antoine et Thomas
- Commandes (vocales ou textuelles) : Yasmine et Nidal
- Configuration : Antoine, Hajar et Yasmine
- Intégration au système : Nidal et Thomas

En effet, la partie Simulation est celle jugée la plus simple, car utilise uniquement le langage Python. Une personne avec des connaissances basiques y est donc confiée.

Concernant les sections Traitement de l'image et Commandes (vocales ou textuelles), des binômes sont constitués d'un membre avec des connaissances basiques, et d'un avec des connaissances plus avancées, afin d'assurer une bonne complémentarité.

La partie Configuration est donnée aux personnes avec des connaissances basiques, qui devront apprendre l'utilisation et les contraintes du format JSON.

Enfin, la partie Intégration au système, jugée la plus difficile, est donc confiée aux membres avec des connaissances avancées en programmation. Cette section permettra par ailleurs d'assurer la bonne cohésion des briques logicielles développées.

Si cette répartition semble équitable, elle ne le sera probablement pas en pratique. Par conséquent, il faut bien noter que celle-ci est modulable : chaque personne peut intervenir dans chaque section en fonction des besoins et des avancées des modules.

Suivi avec le tuteur

Le suivi régulier avec le tuteur constitue un élément essentiel de l'organisation du projet. Des rendez-vous planifiés à intervalles réguliers permettront de présenter l'avancement, de clarifier les attentes et de valider les choix techniques. Ce suivi garantit une bonne adéquation entre les spécifications rédigées, les développements réalisés et les exigences du client, et contribue à limiter les risques d'erreur ou de dérive par rapport aux objectifs fixés.

Comptes rendus

Des comptes rendus sont rédigés à la suite de chaque réunion, qu'elles soient tenues avec le tuteur ou uniquement entre les membres de l'équipe. Ces documents permettent de garder une trace écrite de l'avancement du projet, des décisions prises, des difficultés rencontrées et des actions à mener pour la suite. Ils constituent un support essentiel pour assurer le suivi, la coordination et la bonne progression du projet.

2 Outils et langages

Afin de coordonner le travail, l'équipe s'appuie sur plusieurs outils, partagés avec le tuteur :

- Un dépôt Git pour la gestion de versions et le partage du code.
- Un outil de gestion de tâches tel que Trello pour suivre l'avancement et répartir les responsabilités.

Cette organisation garantit une bonne communication interne et externe, une gestion efficace du temps et une répartition claire des responsabilités, conditions essentielles à la réussite du projet.

Partie Git

Les fichiers seront organisés selon les dossiers suivants dans un dépôt sur [GitHub](#) :

- **bin/** : contient les fichiers exécutables
- **config/** : contient les fichiers de configuration (.json)
- **doc/** : contient la documentation (probablement Doxygen), le manuel d'utilisation (README.txt), le cahier des charges fourni et ce dossier de spécifications
- **image/** : contient les images nécessaires au traitement de l'image (.png, .ppm)
- **include/** : contient les fichiers d'en-tête (.h)
- **lib/** : contient les fichiers de sorties (.o)
- **log/** : contient les fichiers de logs (tests et exécutions)
- **python/** : contient les modules python (.py)
- **src/** : contient les fichiers sources en C (.c)

À la racine du projet se trouve également un fichier **Makefile**, permettant d'automatiser la compilation, la génération des exécutables et la gestion des dépendances entre les fichiers.

Partie Trello

La plateforme [Trello](#) permet de planifier les différentes tâches en incluant des deadline et les personnes concernées. Mais également, les réunions avec le client. Voici une première vue :

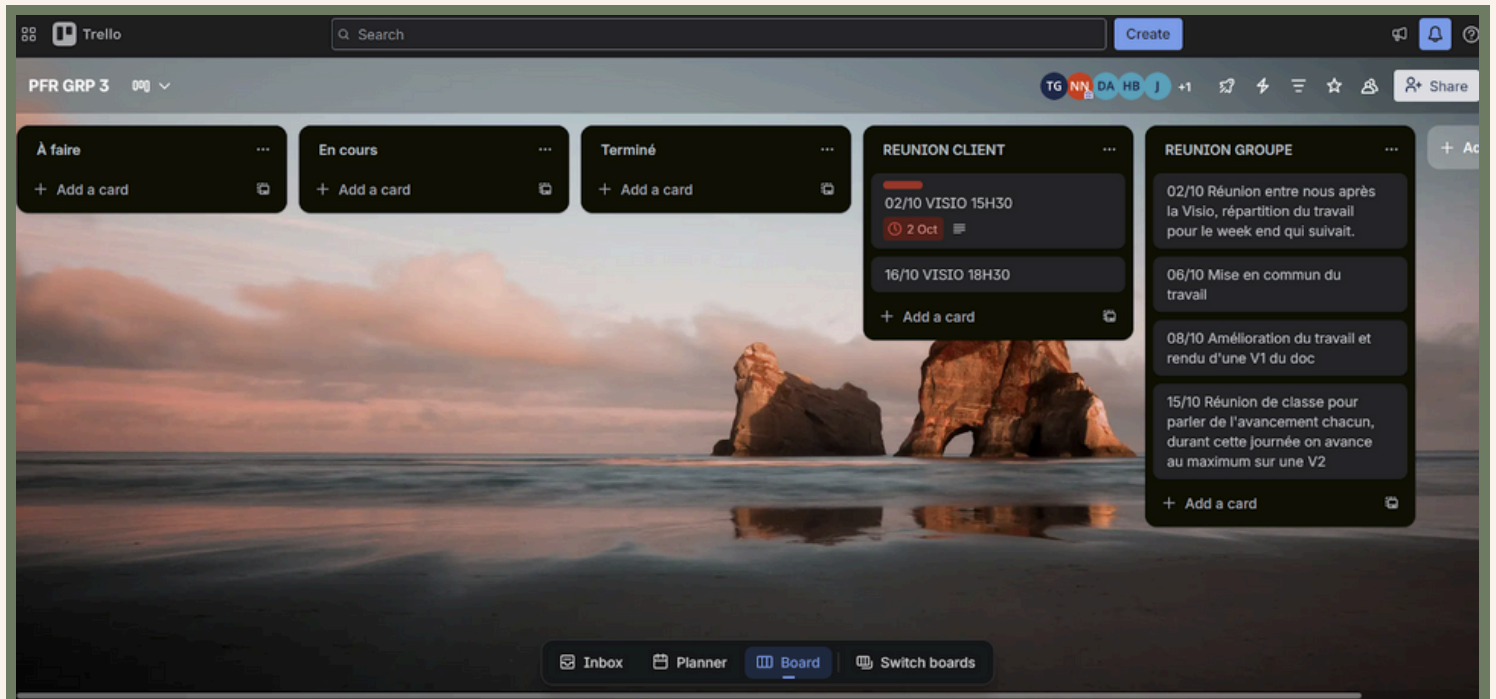


Figure 12 : Vue sur la répartition des tâches sur Trello

3 Améliorations

Afin d'enrichir l'interaction entre l'utilisateur et le système, il serait possible d'intégrer à terme des commandes plus complexes combinant plusieurs actions successives, des conditions logiques ou des changements de mode. Ces améliorations visent à rendre le robot plus autonome et plus réactif à des instructions naturelles, tout en testant la robustesse des modules développés. Le tableau suivant illustre quelques exemples de commandes personnalisées avancées et leur interprétation prévue.

Type de commande	Exemple de commande	Interprétation par le système	Action attendue du robot
Mouvement + orientation	Tourne à droite et fais un demi-tour après avoir avancé de cinq mètres.	Turn_right(90) → Go_forward(5) → Turn_right(180)	Le robot tourne à droite, avance de 5 m, puis effectue une rotation de 180°.
Déplacements conditionnels	Avance jusqu'à ce que tu rencontres un obstacle, puis contourne-le.	Go_forward() en continu + détection obstacle → Avoid_obstacle()	Le robot avance jusqu'à détecter un obstacle, déclenche une action d'évitement, puis continue sa trajectoire.
Commande avec condition	Si tu vois une balle rouge, avance vers elle ; sinon, reste sur place.	Analyse d'image + structure conditionnelle	Si balle rouge détectée → avance ; sinon, aucune action.
Commande avec évitement	Avance de quatre mètres en évitant les obstacles.	Go_forward(4) + activation de l'évitement	Le robot avance et adapte sa trajectoire pour contourner les obstacles.
Commande avec détection d'objet	Cherche la balle rouge à gauche et avance vers elle de trois mètres.	Detect_object(red_ball, left) → Go_forward(3)	Le robot cherche la balle rouge à gauche, la détecte, puis avance de 3 m.
Commande avec changement de mode	Passe en mode automatique après avoir trouvé la balle bleue.	Detect_object(blue_ball) → Switch_mode(auto)	Le robot détecte la balle bleue puis active le mode automatique.
Commande avec détection automatique de langue	Find the red ball.	Détection automatique de la langue anglaise → chargement de la configuration linguistique correspondante → détection de l'objet	Le robot reconnaît que la commande est en anglais, adapte son lexique, puis recherche la balle rouge dans la scène.
Commande trop longue ou complexe	Avance de trois mètres, tourne à gauche, cherche la balle bleue et passe en mode automatique.	Go_forward(3) → Turn_left(90) → Detect_object(blue_ball) → Switch_mode(auto)	Le robot exécute les actions dans l'ordre défini. Si la commande dépasse les capacités prévues, les parties non reconnues sont ignorées.

Tableau 8 : Scénario de commandes personnalisées et interprétation système

Ouverture sur le projet

Ce dossier de spécification sera utile au bon déroulement de la première phase du Projet Fil Rouge, en particulier celle du développement des briques logicielles, nécessaire pour la deuxième phase. Il va permettre à l'équipe d'avancer de manière structurée, en garantissant la cohérence des développements et en facilitant les validations futures.